

LAPORAN TUGAS BESAR 1
IF4021 PEMODELAN DAN SIMULASI

Simulasi Antrian Perjalanan Bus



Disusun oleh:

Ahmad Ibrahim 13523089

Carlo Angkisan 13523091

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG

2026

Daftar Isi

1	Deskripsi Masalah	1
2	Source Code	2
3	Output Program	9

1 Deskripsi Masalah

Permasalahan yang dimodelkan adalah sistem bus *shuttle* yang beroperasi di sebuah bandara dengan tiga lokasi pemberhentian, yaitu Terminal 1, Terminal 2, dan Car Rental. Sistem ini merupakan soal 2.38 pada buku *Simulation Modeling and Analysis* karya Averill M. Law (halaman 179). Bus beroperasi secara berurutan dari Car Rental, kemudian menuju Terminal 1, Terminal 2, dan kembali ke Car Rental. Jarak antar lokasi tersebut secara berturut-turut adalah 4,5 mil, 1 mil, dan 4,5 mil, sehingga dengan kecepatan bus sebesar 30 mil/jam, waktu perjalanan antar lokasi masing-masing adalah 9 menit, 2 menit, dan 9 menit.

Penumpang datang secara acak di setiap lokasi mengikuti proses Poisson dengan laju kedatangan 14 orang/jam di Terminal 1, 10 orang/jam di Terminal 2, dan 24 orang/jam di Car Rental. Penumpang dari Terminal 1 dan Terminal 2 menuju Car Rental, sedangkan penumpang dari Car Rental menuju Terminal 1 dengan probabilitas 0,583 dan Terminal 2 dengan probabilitas 0,417. Bus memiliki kapasitas 20 orang. Waktu untuk menurunkan penumpang mengikuti distribusi $U(16, 24)$ detik, sedangkan waktu untuk menaikkan penumpang mengikuti distribusi $U(15, 25)$ detik. Pada setiap lokasi, bus harus berhenti setidaknya selama 5 menit.

Sistem ini dimodelkan menggunakan *Discrete-Event Simulation* dan disimulasikan selama 80 jam. *Event* dalam simulasi meliputi kedatangan penumpang, kedatangan bus, selesai proses naik dan turun penumpang, tercapainya waktu berhenti minimum, keberangkatan bus, dan akhir simulasi. Hasil simulasi digunakan untuk menganalisis jumlah dan *delay* antrian, jumlah penumpang di dalam bus, waktu berhenti bus, waktu satu *loop*, serta waktu penumpang berada dalam sistem berdasarkan lokasi asal.

Interpretasi Masalah

Karena terdapat beberapa aspek yang tidak dijelaskan secara rinci dalam soal, simulasi ini menggunakan beberapa asumsi dan interpretasi sebagai berikut:

- Bus mulai dari Car Rental pada menit ke-0 dan langsung berangkat tanpa berhenti 5 menit terlebih dahulu. Satu *loop* dihitung sejak bus berangkat dari Car Rental hingga bus berangkat lagi dari Car Rental.
- Saat bus berhenti, penumpang yang tujuannya lokasi tersebut diturunkan terlebih dahulu, baru kemudian penumpang di antrian naik. Proses turun dan naik dilakukan satu per satu sesuai urutan kedatangan.
- Penumpang yang datang ketika bus masih berhenti di lokasinya tetap boleh naik selama bus belum penuh.
- Bus baru berangkat apabila sudah berhenti selama 5 menit, tidak ada lagi penumpang yang turun, dan antrian sudah kosong atau bus sudah penuh. Dengan demikian, jika setelah 5 menit masih ada penumpang yang naik, bus tetap menunggu.
- *Delay* di antrian dihitung sejak penumpang datang hingga ia mulai naik. Sejak mulai naik, penumpang tersebut sudah dihitung berada di dalam bus sampai selesai turun.
- Waktu di sistem dihitung sejak penumpang datang hingga ia selesai turun di lokasi tujuannya.

2 Source Code

```
1  #include "simlib.h"
2
3  #define NUM_LOCATIONS 3
4  #define CAR_RENTAL 3
5
6  #define EVENT_ARRIVAL_1 1
7  #define EVENT_ARRIVAL_2 2
8  #define EVENT_ARRIVAL_3 3
9  #define EVENT_BUS_ARRIVAL 4
10 #define EVENT_UNLOAD_DONE 5
11 #define EVENT_LOAD_DONE 6
12 #define EVENT_MIN_STOP_DONE 7
13 #define EVENT_BUS_DEPARTURE 8
14 #define EVENT_END_SIMULATION 9
15
16 #define LIST_QUEUE(i) (i)
17 #define LIST_BUS(j) (NUM_LOCATIONS + (j))
18
19 #define ATTR_ARRIVAL_TIME 3
20 #define ATTR_ORIGIN 4
21 #define ATTR_DESTINATION 5
22
23 #define SAMPST_DELAY(i) (i)
24 #define SAMPST_STOP(i) (3 + (i))
25 #define SAMPST_LOOP 7
26 #define SAMPST_SYSTEM(i) (7 + (i))
27
28 #define TIMEST_BUS 1
29
30 #define STREAM_ARRIVAL(i) (i)
31 #define STREAM_UNLOAD 4
32 #define STREAM_LOAD 5
33 #define STREAM_DESTINATION 6
34
35 double arrival_rate[NUM_LOCATIONS + 1];
36 double prob_dest[NUM_LOCATIONS];
37 int bus_capacity;
38 double bus_speed;
39 double unload_min, unload_max;
40 double load_min, load_max;
41 double min_stop_time;
42 double sim_length;
43
44 double mean_interarrival[NUM_LOCATIONS + 1];
45
46 /* miles from location i to the next location on the route */
47 double distance_to_next[NUM_LOCATIONS + 1] = {0.0, 1.0, 4.5, 4.5};
48
49 int bus_location;
50 int bus_at_stop;
51 int bus_busy;
52 int min_stop_passed;
53 double bus_stop_start;
54 double loop_start;
55
56 FILE *infile, *outfile;
```

```

57
58 void init_model(void);
59 void arrive(int location);
60 void bus_arrive(void);
61 void unload_done(void);
62 void load_done(void);
63 void min_stop_done(void);
64 void bus_depart(void);
65 void process_bus_at_stop(void);
66 int num_on_bus(void);
67 void print_stat(const char *tag, const char *label, double avg, double max, double min
    , int is_count, int has_min);
68 void write_report(void);
69 void report(void);
70
71 int main()
72 {
73     infile = fopen("data/bus.in", "r");
74     if (infile == NULL)
75     {
76         fprintf(stderr, "Failed to open data/bus.in\n");
77         exit(1);
78     }
79
80     fscanf(infile, "%lf %lf %lf", &arrival_rate[1], &arrival_rate[2], &arrival_rate
        [3]);
81     fscanf(infile, "%lf %lf", &prob_dest[1], &prob_dest[2]);
82     fscanf(infile, "%d %lf", &bus_capacity, &bus_speed);
83     fscanf(infile, "%lf %lf", &unload_min, &unload_max);
84     fscanf(infile, "%lf %lf", &load_min, &load_max);
85     fscanf(infile, "%lf", &min_stop_time);
86     fscanf(infile, "%lf", &sim_length);
87
88     for (int i = 1; i <= NUM_LOCATIONS; ++i)
89         mean_interarrival[i] = 60.0 / arrival_rate[i];
90
91     /* random_integer needs a cumulative distribution */
92     prob_distrib[1] = prob_dest[1];
93     prob_distrib[2] = prob_dest[1] + prob_dest[2];
94
95     maxatr = 5;
96     init_simlib();
97
98     init_model();
99
100 do
101 {
102     timing();
103
104     switch (next_event_type)
105     {
106     case EVENT_ARRIVAL_1:
107     case EVENT_ARRIVAL_2:
108     case EVENT_ARRIVAL_3:
109         arrive(next_event_type);
110         break;
111     case EVENT_BUS_ARRIVAL:
112         bus_arrive();

```

```

113         break;
114     case EVENT_UNLOAD_DONE:
115         unload_done();
116         break;
117     case EVENT_LOAD_DONE:
118         load_done();
119         break;
120     case EVENT_MIN_STOP_DONE:
121         min_stop_done();
122         break;
123     case EVENT_BUS_DEPARTURE:
124         bus_depart();
125         break;
126     case EVENT_END_SIMULATION:
127         report();
128         break;
129     }
130 } while (next_event_type != EVENT_END_SIMULATION);
131
132 fclose(infile);
133
134 return 0;
135 }
136
137 void init_model(void)
138 {
139     bus_location = CAR_RENTAL;
140     bus_at_stop = 0;
141     bus_busy = 0;
142     min_stop_passed = 0;
143     bus_stop_start = 0.0;
144     loop_start = 0.0;
145
146     for (int i = 1; i <= NUM_LOCATIONS; ++i)
147         event_schedule(expon(mean_interarrival[i], STREAM_ARRIVAL(i)), EVENT_ARRIVAL_1
148             + i - 1);
149
150     event_schedule(0.0, EVENT_BUS_DEPARTURE);
151     event_schedule(sim_length * 60.0, EVENT_END_SIMULATION);
152 }
153
154 void arrive(int location)
155 {
156     event_schedule(sim_time + expon(mean_interarrival[location], STREAM_ARRIVAL(
157         location)),
158         EVENT_ARRIVAL_1 + location - 1);
159
160     transfer[ATTR_ARRIVAL_TIME] = sim_time;
161     transfer[ATTR_ORIGIN] = location;
162     if (location == CAR_RENTAL)
163         transfer[ATTR_DESTINATION] = random_integer(prob_distrib, STREAM_DESTINATION);
164     else
165         transfer[ATTR_DESTINATION] = CAR_RENTAL;
166     list_file(LAST, LIST_QUEUE(location));
167
168     if (bus_at_stop && bus_location == location)
169         process_bus_at_stop();
170 }

```

```

169
170 void bus_arrive(void)
171 {
172     bus_at_stop = 1;
173     bus_stop_start = sim_time;
174     min_stop_passed = 0;
175     event_schedule(sim_time + min_stop_time, EVENT_MIN_STOP_DONE);
176
177     process_bus_at_stop();
178 }
179
180 void unload_done(void)
181 {
182     list_remove(FIRST, LIST_BUS(bus_location));
183     sampst(sim_time - transfer[ATTR_ARRIVAL_TIME], SAMPST_SYSTEM((int)transfer[
ATTR_ORIGIN]));
184     timest(num_on_bus(), TIMEST_BUS);
185
186     bus_busy = 0;
187     process_bus_at_stop();
188 }
189
190 void load_done(void)
191 {
192     bus_busy = 0;
193     process_bus_at_stop();
194 }
195
196 void min_stop_done(void)
197 {
198     min_stop_passed = 1;
199     process_bus_at_stop();
200 }
201
202 void bus_depart(void)
203 {
204     /* the first departure at t = 0 has no stop or loop to record */
205     if (sim_time > 0.0)
206         sampst(sim_time - bus_stop_start, SAMPST_STOP(bus_location));
207
208     if (bus_location == CAR_RENTAL)
209     {
210         if (sim_time > 0.0)
211             sampst(sim_time - loop_start, SAMPST_LOOP);
212         loop_start = sim_time;
213     }
214
215     event_schedule(sim_time + distance_to_next[bus_location] / bus_speed * 60.0,
EVENT_BUS_ARRIVAL);
216     bus_location = bus_location % NUM_LOCATIONS + 1;
217 }
218
219 void process_bus_at_stop(void)
220 {
221     if (bus_busy)
222         return;
223
224     if (list_size[LIST_BUS(bus_location)] > 0)

```

```

225     {
226         bus_busy = 1;
227         event_schedule(sim_time + uniform(unload_min, unload_max, STREAM_UNLOAD) /
60.0, EVENT_UNLOAD_DONE);
228     }
229     else if (list_size[LIST_QUEUE(bus_location)] > 0 && num_on_bus() < bus_capacity)
230     {
231         list_remove(FIRST, LIST_QUEUE(bus_location));
232         sampst(sim_time - transfer[ATTR_ARRIVAL_TIME], SAMPST_DELAY(bus_location));
233         list_file(LAST, LIST_BUS((int)transfer[ATTR_DESTINATION]));
234         timest(num_on_bus(), TIMEST_BUS);
235
236         bus_busy = 1;
237         event_schedule(sim_time + uniform(load_min, load_max, STREAM_LOAD) / 60.0,
EVENT_LOAD_DONE);
238     }
239     else if (min_stop_passed)
240     {
241         bus_at_stop = 0;
242         event_schedule(sim_time, EVENT_BUS_DEPARTURE);
243     }
244 }
245
246 int num_on_bus(void)
247 {
248     int total = 0;
249
250     for (int j = 1; j <= NUM_LOCATIONS; ++j)
251         total += list_size[LIST_BUS(j)];
252     return total;
253 }
254
255 void print_stat(const char *tag, const char *label, double avg, double max, double min
, int is_count, int has_min)
256 {
257     char max_str[16], min_str[16] = "-";
258
259     snprintf(max_str, sizeof max_str, is_count ? "%.0f" : "%.3f", max);
260     if (has_min)
261         snprintf(min_str, sizeof min_str, "%.3f", min);
262     fprintf(outfile, "%-4s%-38s|10.3f |10s |10s\n", tag, label, avg, max_str,
min_str);
263 }
264
265 void report(void)
266 {
267     char answer[8];
268
269     outfile = stdout;
270     write_report();
271
272     printf("\nSave output to data/bus.out? (y/n): ");
273     if (fgets(answer, sizeof answer, stdin) == NULL || (answer[0] != 'y' && answer[0]
!= 'Y'))
274         return;
275
276     outfile = fopen("data/bus.out", "w");
277     if (outfile == NULL)

```

```

278     {
279         fprintf(stderr, "Failed to open data/bus.out\n");
280         return;
281     }
282     write_report();
283     fclose(outfile);
284     printf("Output saved to data/bus.out\n");
285 }
286
287 void write_report(void)
288 {
289     char label[64];
290
291     fprintf(outfile, "AIRPORT SHUTTLE BUS SIMULATION\n\n");
292
293     fprintf(outfile, "Input Parameters\n");
294     fprintf(outfile, "-----\n");
295     fprintf(outfile, "%-42s= %.1f %.1f %.1f per hour\n", "Arrival rates at locations
296 1, 2, 3", arrival_rate[1], arrival_rate[2], arrival_rate[3]);
297     fprintf(outfile, "%-42s= %.3f %.3f\n", "Destination probabilities from car rental
298 , prob_dest[1], prob_dest[2]);
299     fprintf(outfile, "%-42s= %d people\n", "Bus capacity", bus_capacity);
300     fprintf(outfile, "%-42s= %.1f miles per hour\n", "Bus speed", bus_speed);
301     fprintf(outfile, "%-42s= %.1f to %.1f seconds\n", "Unloading time per person",
302 unload_min, unload_max);
303     fprintf(outfile, "%-42s= %.1f to %.1f seconds\n", "Loading time per person",
304 load_min, load_max);
305     fprintf(outfile, "%-42s= %.1f minutes\n", "Minimum stop time", min_stop_time);
306     fprintf(outfile, "%-42s= %.1f hours\n\n", "Simulation length", sim_length);
307
308     fprintf(outfile, "Simulation Results\n");
309     fprintf(outfile, "-----\n");
310     fprintf(outfile, "Locations: 1 = Terminal 1, 2 = Terminal 2, 3 = Car rental\n");
311     fprintf(outfile, "All times are in minutes\n\n");
312     fprintf(outfile, "%-42s|%10s |%10s |%10s\n", "", "Average", "Max", "Min");
313
314     for (int i = 1; i <= NUM_LOCATIONS; ++i)
315     {
316         filest(LIST_QUEUE(i));
317         snprintf(label, sizeof label, "Number in queue at location %d", i);
318         print_stat(i == 1 ? "[a]" : "", label, transfer[1], transfer[2], 0.0, 1, 0);
319     }
320
321     for (int i = 1; i <= NUM_LOCATIONS; ++i)
322     {
323         sampst(0.0, -SAMPST_DELAY(i));
324         snprintf(label, sizeof label, "Delay in queue at location %d", i);
325         print_stat(i == 1 ? "[b]" : "", label, transfer[1], transfer[3], 0.0, 0, 0);
326     }
327
328     timest(0.0, -TIMEST_BUS);
329     print_stat("[c]", "Number on the bus", transfer[1], transfer[2], 0.0, 1, 0);
330
331     for (int i = 1; i <= NUM_LOCATIONS; ++i)
332     {
333         sampst(0.0, -SAMPST_STOP(i));
334         snprintf(label, sizeof label, "Bus stop time at location %d", i);

```

```

331     print_stat(i == 1 ? "[d]" : "", label, transfer[1], transfer[3], transfer[4],
332     0, 1);
333 }
334 sampst(0.0, -SAMPST_LOOP);
335 print_stat("[e]", "Bus loop time", transfer[1], transfer[3], transfer[4], 0, 1);
336
337 for (int i = 1; i <= NUM_LOCATIONS; ++i)
338 {
339     sampst(0.0, -SAMPST_SYSTEM(i));
340     snprintf(label, sizeof label, "Time in system, arrived at location %d", i);
341     print_stat(i == 1 ? "[f]" : "", label, transfer[1], transfer[3], transfer[4],
342     0, 1);
343 }

```

3 Output Program

AIRPORT SHUTTLE BUS SIMULATION

Input Parameters

```

-----
Arrival rates at locations 1, 2, 3      = 14.0 10.0 24.0 per hour
Destination probabilities from car rental = 0.583 0.417
Bus capacity                            = 20 people
Bus speed                               = 30.0 miles per hour
Unloading time per person                = 16.0 to 24.0 seconds
Loading time per person                  = 15.0 to 25.0 seconds
Minimum stop time                       = 5.0 minutes
Simulation length                        = 80.0 hours

```

Simulation Results

```

-----
Locations: 1 = Terminal 1, 2 = Terminal 2, 3 = Car rental
All times are in minutes

```

	Average	Max	Min
[a] Number in queue at location 1	7.368	27	-
Number in queue at location 2	4.888	17	-
Number in queue at location 3	9.321	29	-
[b] Delay in queue at location 1	31.414	91.934	-
Delay in queue at location 2	29.820	130.860	-
Delay in queue at location 3	24.130	67.485	-
[c] Number on the bus	14.114	20	-
[d] Bus stop time at location 1	6.698	10.171	5.000
Bus stop time at location 2	5.473	9.287	5.000
Bus stop time at location 3	11.402	13.743	5.572
[e] Bus loop time	43.573	48.069	35.715
[f] Time in system, arrived at location 1	51.927	112.294	17.738
Time in system, arrived at location 2	45.544	147.004	13.929
Time in system, arrived at location 3	41.724	88.289	11.532